

OUTPUT SCREENSHOTS

- Understand dataset features and target variable.

```
[2]: #3))-----view dataset
df.head()
df.tail()
df.sample(10)
```

[2]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	..	
	710	33	No	Non-Travel	722	Sales	17	3	Life Sciences	1	992	..
	1395	31	Yes	Travel_Frequently	754	Sales	26	4	Marketing	1	1967	..
	1127	23	No	Travel_Rarely	977	Research & Development	10	3	Technical Degree	1	1592	..
	1228	41	No	Non-Travel	552	Human Resources	4	3	Human Resources	1	1722	..
	1115	34	No	Travel_Rarely	479	Research & Development	7	4	Medical	1	1577	..
	1203	53	No	Travel_Rarely	1395	Research & Development	24	4	Medical	1	1689	..
	749	52	Yes	Travel_Rarely	266	Sales	2	1	Marketing	1	1038	..
	114	34	No	Travel_Rarely	1031	Research & Development	6	4	Life Sciences	1	151	..

```
#4))-----dataset shape
df.shape
```

[4]: (1470, 35)

```
[5]: #5))-----dataset information
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                Non-Null Count  Dtype
---  ---
0    Age                   1470 non-null  int64
1    Attrition             1470 non-null  str
2    BusinessTravel        1470 non-null  str
3    DailyRate             1470 non-null  int64
4    Department            1470 non-null  str
5    DistanceFromHome      1470 non-null  int64
6    Education              1470 non-null  int64
7    EducationField         1470 non-null  str
8    EmployeeCount          1470 non-null  int64
9    EmployeeNumber         1470 non-null  int64
10   EnvironmentSatisfaction 1470 non-null  int64
11   Gender                1470 non-null  str
12   HourlyRate            1470 non-null  int64
13   JobInvolvement         1470 non-null  int64
14   JobLevel               1470 non-null  int64
15   JobRole                1470 non-null  str
16   JobSatisfaction        1470 non-null  int64
17   MaritalStatus          1470 non-null  str
```

15	JobRole	1470 non-null	str
16	JobSatisfaction	1470 non-null	int64
17	MaritalStatus	1470 non-null	str
18	MonthlyIncome	1470 non-null	int64
19	MonthlyRate	1470 non-null	int64
20	NumCompaniesWorked	1470 non-null	int64
21	Over18	1470 non-null	str
22	OverTime	1470 non-null	str
23	PercentsSalaryHike	1470 non-null	int64
24	PerformanceRating	1470 non-null	int64
25	RelationshipSatisfaction	1470 non-null	int64
26	StandardHours	1470 non-null	int64
27	StockOptionLevel	1470 non-null	int64
28	TotalWorkingYears	1470 non-null	int64
29	TrainingTimesLastYear	1470 non-null	int64
30	WorkLifeBalance	1470 non-null	int64
31	YearsAtCompany	1470 non-null	int64
32	YearsInCurrentRole	1470 non-null	int64
33	YearsSincelastPromotion	1470 non-null	int64
34	YearsWithCurrManager	1470 non-null	int64

dtypes: int64(26), str(9)
memory usage: 402.1 KB

```
#6)-----dataset columns
```

```
df.columns
```

```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',  
      'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',  
      'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',  
      'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',  
      'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',  
      'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',  
      'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',  
      'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',  
      'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',  
      'YearsWithCurrManager'],  
      dtype='str')
```

```
[7]: Age                int64  
      Attrition          str  
      BusinessTravel     str  
      DailyRate          int64  
      Department         str  
      DistanceFromHome   int64  
      Education          int64  
      EducationField     str  
      EmployeeCount      int64  
      EmployeeNumber     int64  
      EnvironmentSatisfaction int64  
      Gender            str  
      HourlyRate         int64  
      JobInvolvement     int64  
      JobLevel           int64  
      JobRole            str  
      JobSatisfaction    int64  
      MaritalStatus      str  
      MonthlyIncome      int64  
      MonthlyRate        int64  
      NumCompaniesWorked int64  
      Over18            str  
      OverTime           str  
      PercentSalaryHike  int64  
      PerformanceRating  int64  
      RelationshipSatisfaction int64  
      StandardHours      int64  
      StockOptionLevel   int64  
      TotalWorkingYears  int64  
      TrainingTimesLastYear int64  
      WorkLifeBalance    int64
```

```
[10]: #1)----- checking and handling how many missing values
df.isnull().sum()
```

```
[10]: Age                                0
Attrition                              0
BusinessTravel                         0
DailyRate                             0
Department                             0
DistanceFromHome                       0
Education                             0
EducationField                         0
EmployeeCount                          0
EmployeeNumber                         0
EnvironmentSatisfaction                0
Gender                                 0
HourlyRate                             0
JobInvolvement                         0
JobLevel                               0
JobRole                                0
JobSatisfaction                        0
MaritalStatus                          0
MonthlyIncome                         0
MonthlyRate                           0
NumCompaniesWorked                    0
Over18                                 0
OverTime                               0
PercentSalaryHike                     0
PerformanceRating                     0
RelationshipSatisfaction               0
StandardHours                         0
StockOptionLevel                      0
```

```
#4) -----FEATURE SCALING
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
pd.DataFrame(X).head()
```

	0	1	2	3	4	5	6	7	8	9	...	24	25	26	27	
0	0.446350	0.590048	0.742527	1.401512	-1.010909	-0.891688	-0.937414	0.0	-1.701283	-0.660531	...	-1.584178	0.0	-0.932014	-0.421642	-2.1
1	1.322365	-0.913194	-1.297775	-0.493817	-0.147150	-1.868426	-0.937414	0.0	-1.699621	0.254625	...	1.191438	0.0	0.241988	-0.164511	0.1
2	0.008343	0.590048	1.414363	-0.493817	-0.887515	-0.891688	1.316673	0.0	-1.696298	1.169781	...	-0.658973	0.0	-0.932014	-0.550208	0.1
3	-0.429664	-0.913194	1.461466	-0.493817	-0.764121	1.061787	-0.937414	0.0	-1.694636	1.169781	...	0.266233	0.0	-0.932014	-0.421642	0.1
4	-1.086676	0.590048	-0.524295	-0.493817	-0.887515	-1.868426	0.565311	0.0	-1.691313	-1.575686	...	1.191438	0.0	0.241988	-0.678774	0.1

5 rows × 34 columns

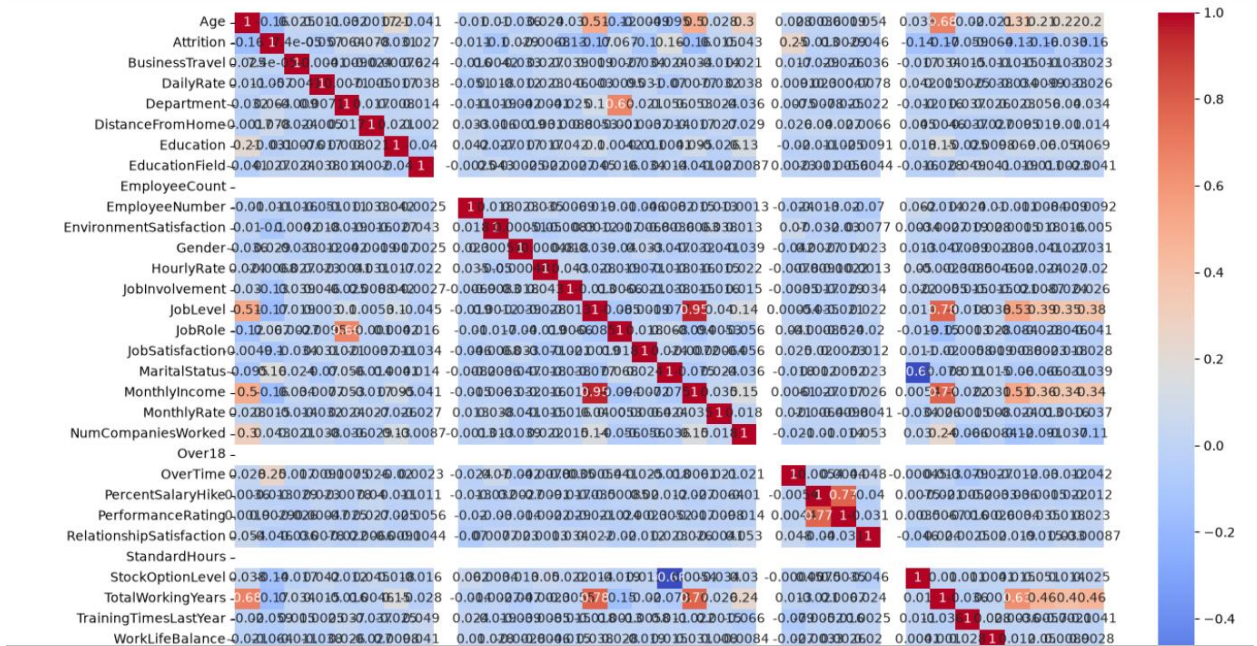
```
#1)-----summary statistics:mathematical computations
df.describe()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Emple
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	
mean	36.923810	0.161224	1.607483	802.485714	1.260544	9.192517	2.912925	2.247619	1.0	
std	9.135373	0.367863	0.665455	403.509100	0.527792	8.106864	1.024165	1.331369	0.0	
min	18.000000	0.000000	0.000000	102.000000	0.000000	1.000000	1.000000	0.000000	1.0	
25%	30.000000	0.000000	1.000000	465.000000	1.000000	2.000000	2.000000	1.000000	1.0	
50%	36.000000	0.000000	2.000000	802.000000	1.000000	7.000000	3.000000	2.000000	1.0	
75%	43.000000	0.000000	2.000000	1157.000000	2.000000	14.000000	4.000000	3.000000	1.0	
max	60.000000	1.000000	2.000000	1499.000000	2.000000	29.000000	5.000000	5.000000	1.0	

8 rows × 35 columns

```
#2)----- correlation matrix to check relation between features
corr= df.corr(numeric_only=True)
corr
```

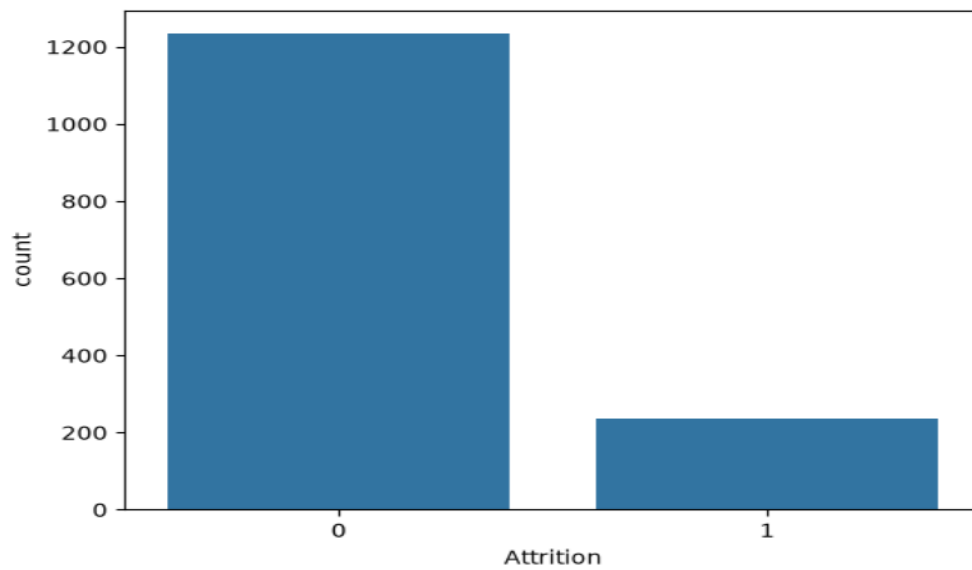
	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount
Age	1.000000	-0.159205	0.024751	0.010661	-0.031882	-0.001686	0.208034	-0.040873	NaN
Attrition	-0.159205	1.000000	0.000074	-0.056652	0.063991	0.077924	-0.031373	0.026846	NaN
BusinessTravel	0.024751	0.000074	1.000000	-0.004086	-0.009044	-0.024469	0.000757	0.023724	NaN
DailyRate	0.010661	-0.056652	-0.004086	1.000000	0.007109	-0.004985	-0.016806	0.037709	NaN
Department	-0.031882	0.063991	-0.009044	0.007109	1.000000	0.017225	0.007996	0.013720	NaN
DistanceFromHome	-0.001686	0.077924	-0.024469	-0.004985	0.017225	1.000000	0.021042	0.002013	NaN
Education	0.208034	-0.031373	0.000757	-0.016806	0.007996	0.021042	1.000000	-0.039592	NaN
EducationField	-0.040873	0.026846	0.023724	0.037709	0.013720	0.002013	-0.039592	1.000000	NaN
EmployeeCount	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
EmployeeNumber	-0.010145	-0.010577	-0.015578	-0.050990	-0.010895	0.032916	0.042070	-0.002516	NaN
EnvironmentSatisfaction	0.010146	-0.103369	0.004174	0.018355	-0.019395	-0.016075	-0.027128	0.043163	NaN
Gender	-0.036311	0.029453	-0.032981	-0.011716	-0.041583	-0.001851	-0.016547	-0.002504	NaN
HourlyRate	0.024287	-0.006846	0.026528	0.023381	-0.004144	0.031131	0.016775	-0.021941	NaN



[27]:

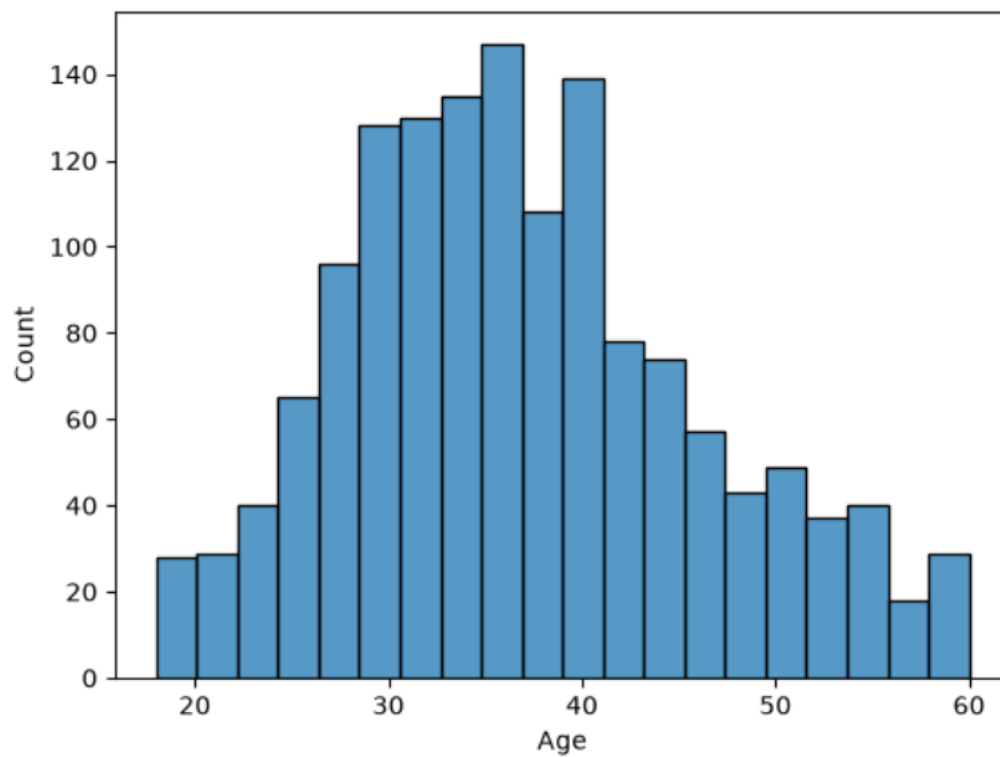
```
#4)-----TARGET DISTRIBUTION  
sns.countplot(x="Attrition",data=df)
```

[27]: <Axes: xlabel='Attrition', ylabel='count'>



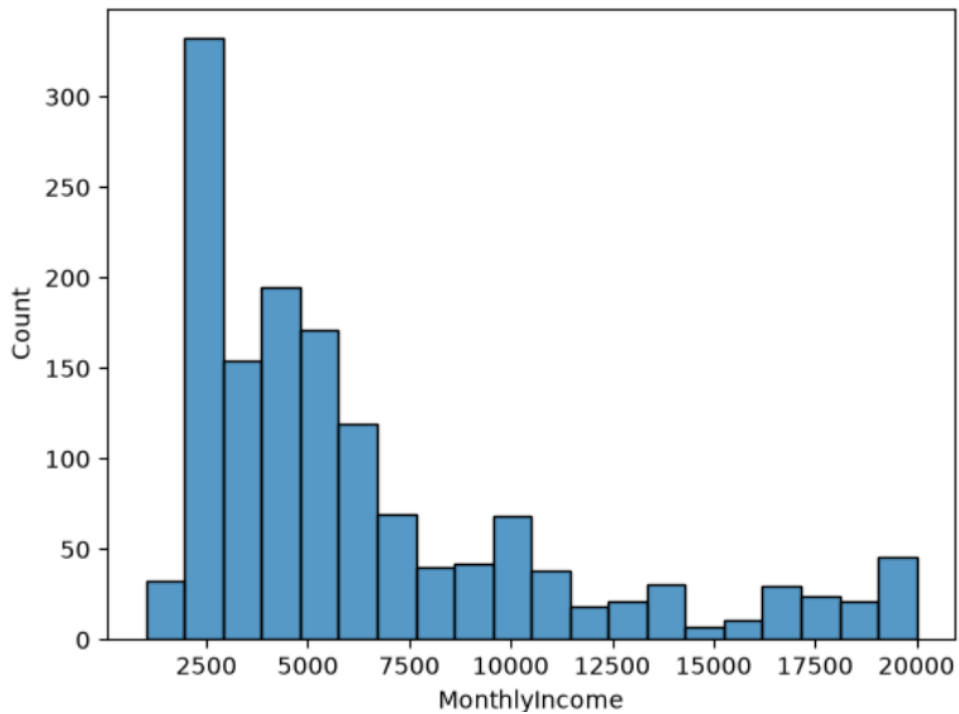
```
sns.histplot(df["Age"],bins=20)
```

[28]: <Axes: xlabel='Age', ylabel='Count'>



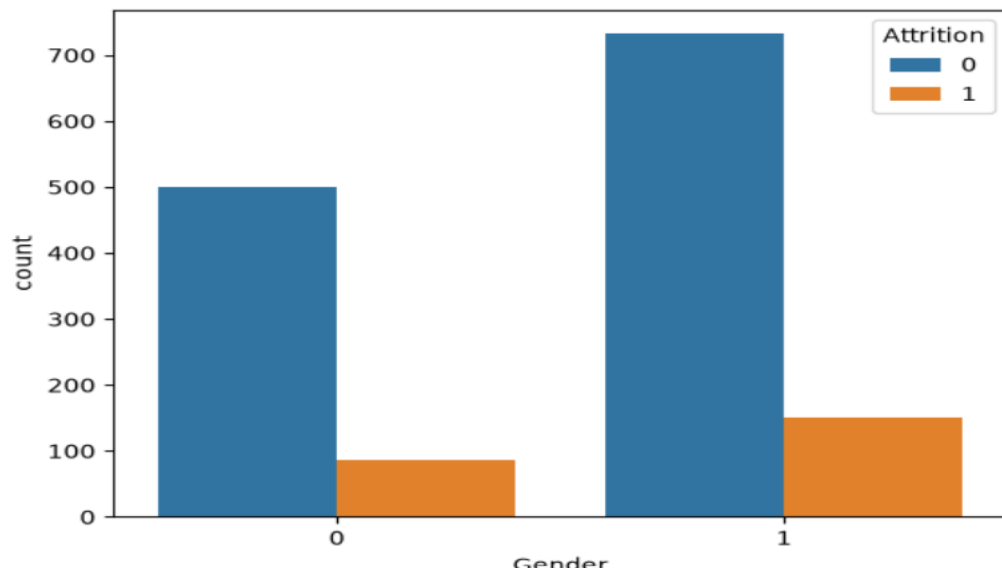
```
[29]: #6)-----MONTHLY INCOME  
sns.histplot(df["MonthlyIncome"])
```

```
[29]: <Axes: xlabel='MonthlyIncome', ylabel='Count'>
```



```
[30]: #6) GENDER VERSUS ATTRITION  
sns.countplot(x="Gender",  
             hue="Attrition",  
             data=df)
```

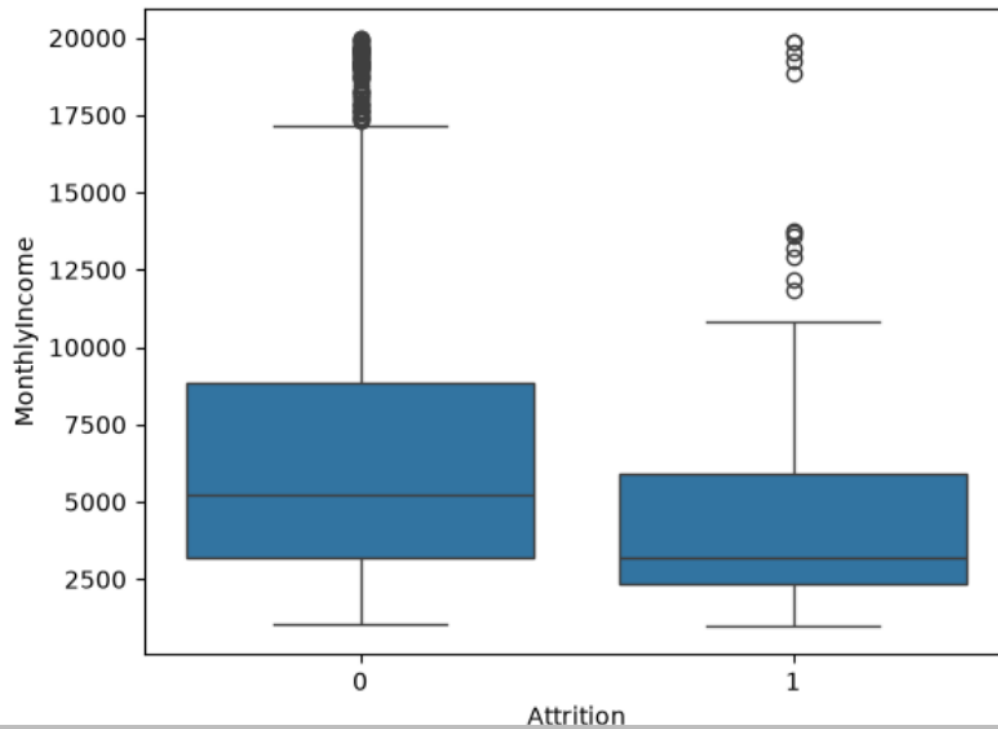
```
[30]: <Axes: xlabel='Gender', ylabel='count'>
```



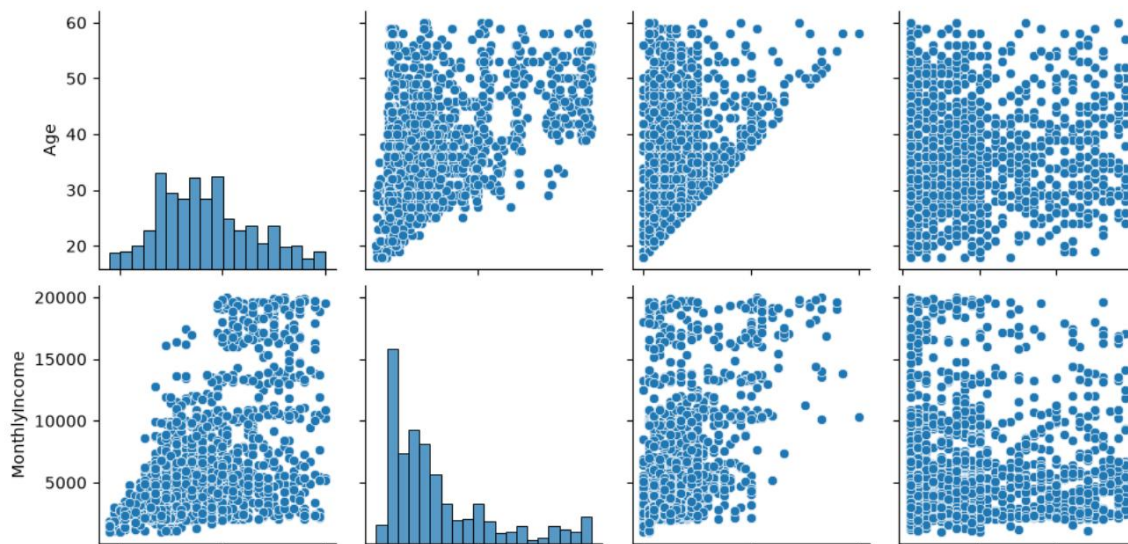
```
#9)----- TO CREATE A BOXPLOT
```

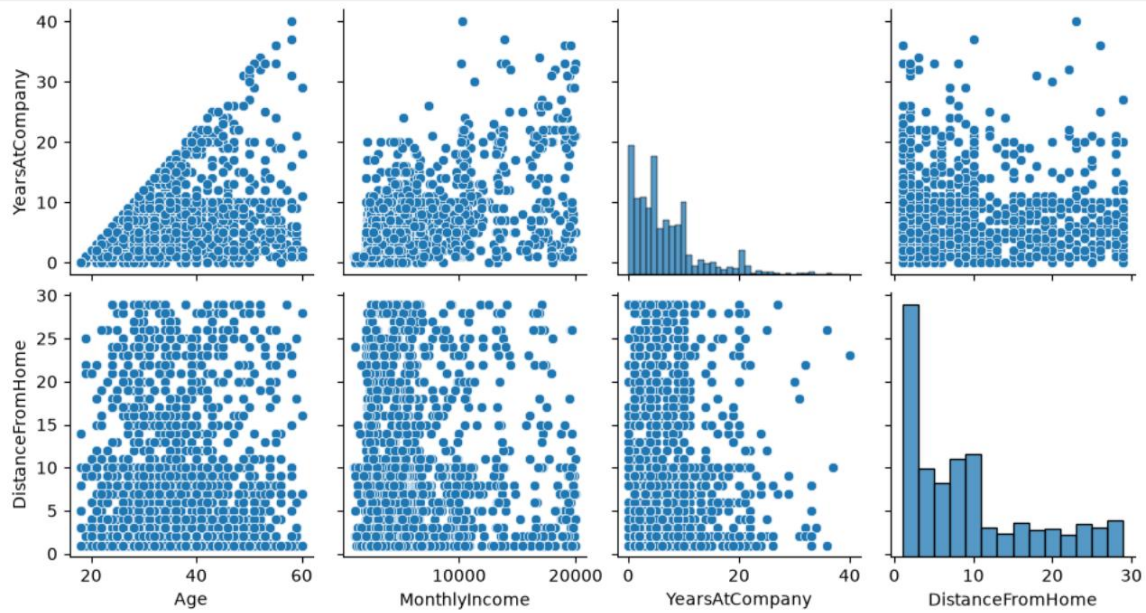
```
sns.boxplot(x="Attrition",  
y="MonthlyIncome",  
data=df)
```

```
[33]: <Axes: xlabel='Attrition', ylabel='MonthlyIncome'>
```



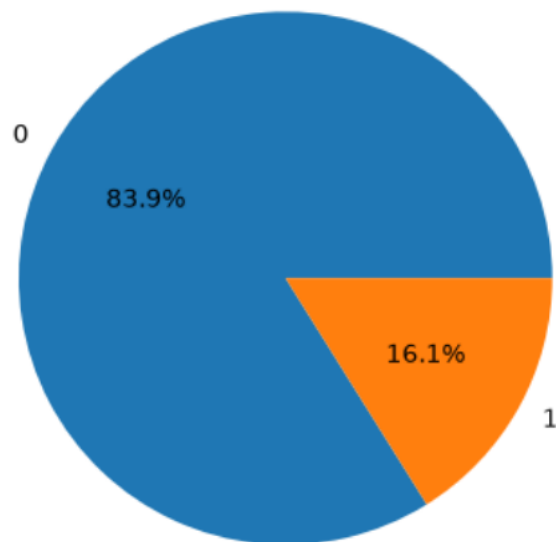
```
[34]: <seaborn.axisgrid.PairGrid at 0x131d2a26510>
```





```
i]: #11)-----EXPLORE DATA TO UNDERSTAND ATTRITION
df["Attrition"].value_counts().plot.pie(autopct="%1.1f%%")
```

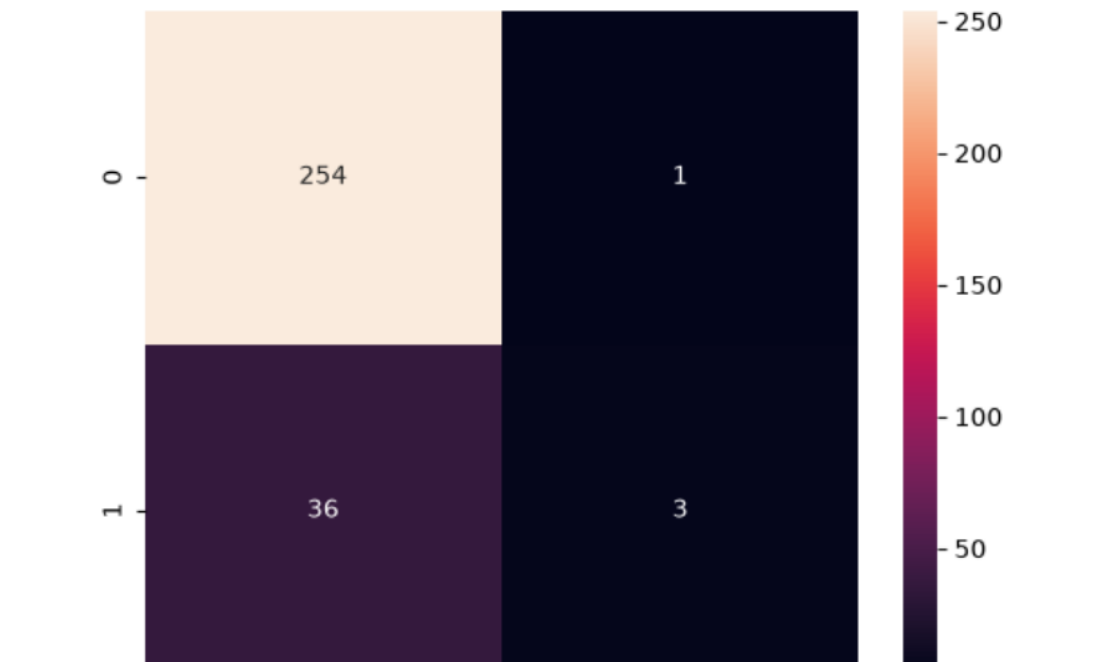
```
i]: <Axes: >
```




```
[47]: cm=confusion_matrix(y_test,pred_rf)

sns.heatmap(cm,
annot=True,
fmt='d')
```

[47]: <Axes: >



- Display Precision, Recall and F1 Score

```
•[48]: #----- PRECISION SCORE-----

from sklearn.metrics import precision_score
precision_score(y_test,pred_rf,pos_label=1)
```

[48]: 0.75

```
[49]: #-----RECALL SCORE-----

from sklearn.metrics import recall_score
recall_score(y_test,pred_rf,pos_label=1)
```

[49]: 0.07692307692307693

```
[50]: #-----F1 SCORE-----

from sklearn.metrics import f1_score
f1_score(y_test,pred_rf,pos_label=1)
```

[50]: 0.13953488372093023

